CS-233 Milestone 1 – Predicting Addiction from Gaming Habits

372055, 384781, 397345

1 Introduction

The dataset is a 2,000-sample subset of the Gaming and Mental Health Dataset. Each sample has 13 features and two targets: an addiction score in $[0, 10]$ for regression, and a discretised addiction level in {Low, Medium, High}, which we treat as the class labels $\{0, 1, 2\}$. The split we were given puts 1,600 samples in training and 400 in the test set. We implement three methods from scratch: closed-form linear regression, multi-class logistic regression with gradient descent, and k-Nearest Neighbors. Hyperparameters are picked on an 80/20 train/validation split of the training data.

2 Method

Pre-processing. Every feature is standardised with the training-set mean and standard deviation, and we reuse those same statistics on validation and test data (no leakage). For the linear models we then add a column of ones to fold the bias into W, so the weight matrix has $D+1$ rows. KNN uses the features without the bias column, since a constant coordinate does not change Euclidean distances.

Linear regression. Closed-form least squares via the Moore–Penrose pseudoinverse:

$$\mathbf{w}^\star = X^+\mathbf{y},$$

computed with `np.linalg.pinv`. Using the pseudoinverse sidesteps the issue of a singular or badly conditioned $X^\top X$ when features are correlated, without having to pick a regularisation strength.

Logistic regression. Standard softmax setup with one weight vector per class, stacked in $W \in \mathbb{R}^{(D+1)\times C}$ and initialised from $\mathcal{N}(0, 0.1^2)$. The loss is multi-class cross-entropy and its gradient simplifies to

$$\nabla_W \mathcal{L} \;=\; \tfrac{1}{N}\, X^\top (P - Y),$$

with P the softmax probabilities and Y the one-hot labels. We train by full-batch gradient descent at a fixed learning rate. The softmax subtracts the row-wise maximum before exponentiating, which keeps things numerically stable for large logits.

k-Nearest Neighbors. For each test point we compute squared Euclidean distances to every training point in one shot, using

$$\|\mathbf{x} - \mathbf{y}\|^2 \;=\; \|\mathbf{x}\|^2 + \|\mathbf{y}\|^2 - 2\,\mathbf{x}^\top\mathbf{y}.$$

The k nearest indices come from `np.argpartition`, which is $\mathcal{O}(N)$ rather than $\mathcal{O}(N \log N)$ because we do not need a full sort. Classification one-hot encodes the k neighbour labels, sums them, and takes the arg max, which is just a majority vote written vectorially. Regression averages the neighbour targets. We skip the square root because it does not change the ordering of the distances.

3 Experiments and Results

Hyperparameter selection. We carve out 20% of the training set as a validation split (seed 100, fixed across all runs) and sweep each method's hyperparameter. The value that gives the best validation metric is the one we keep. Figure 1 shows the sweeps.

Selected hyperparameters. For KNN classification we pick $K = 10$. Validation accuracy is basically flat between $K = 5$ and $K = 10$ (around 75%), but $K = 10$ is the most stable point before macro-F1 starts collapsing. KNN regression: $K = 10$ again, the clear minimum of the MSE curve at 1.91. Logistic regression: lr $= 0.1$ with `max_iters` $= 1000$. Smaller learning rates are not wrong, they just need a lot more iterations to get to the same place. Linear regression has no hyperparameters to choose.

Final test-set results. With hyperparameters fixed, we retrain each model on the full training set (the `--test` flag) and evaluate on the real test set. Table 1 gives the numbers.

Classification	Acc. (%)	Macro-F1
Logistic reg. (lr=0.1, iters=1000)	**88.25**	**0.819**
KNN (K=10)	79.75	0.510
Regression	MSE	
Linear regression	**1.001**	
KNN (K=10)	1.849	

Table 1: Test-set metrics after retraining on the full training set.

4 Discussion and Conclusion

On classification, logistic regression wins by a large margin: 88.25% test accuracy and 0.82 macro-F1, against 79.75% and 0.51 for KNN at $K = 10$. The F1 gap is the telling one. KNN gets respectable accuracy mostly because the majority class is common, but its per-class performance is weak, which is what the low macro-F1 picks up. A linear decision boundary in the standardised feature space seems to fit this data much better than local neighbour voting does.

The regression story is similar. Linear regression hits a test MSE of 1.001, KNN at $K = 10$ stops at 1.849, almost twice as bad. Both results point to the same thing: the target is roughly linear in the features we have, and that is the setting where linear models are hard to beat. KNN only really pays off when local geometry carries information a global linear fit cannot see, and there is not much of that here.

A note on the train/test gap. The numbers in Table 1 come with one surprise. Logistic regression scores 85.7% accuracy on the training set but 88.3% on the test set. Test-above-train is unusual; it usually points to the test split being slightly easier (a smaller proportion of hard boundary cases), or to the validation-set size being small

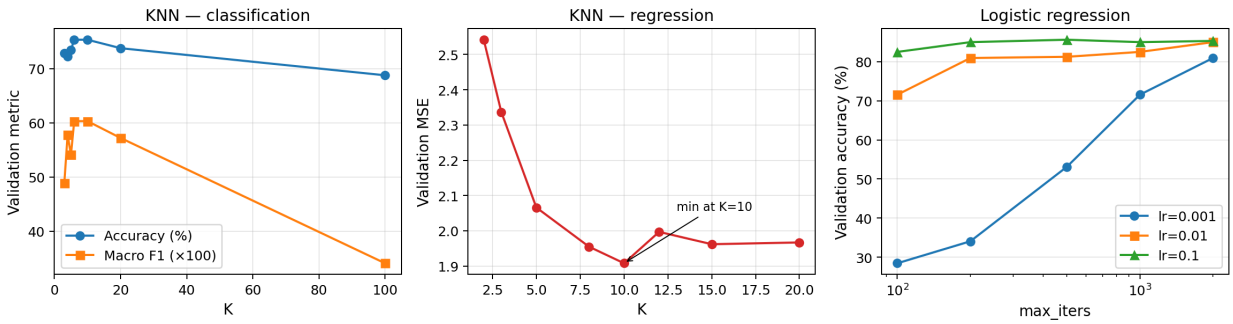

Figure 1: Validation performance against each hyperparameter. **Left:** KNN classification. Accuracy and macro-F1 both peak at K=10 (around 75%), then drop as K grows. **Middle:** KNN regression. MSE bottoms out at K=10 (1.91); below that it overfits, above that it under-fits. **Right:** logistic regression. At lr = 0.1 accuracy saturates within a few hundred iterations; smaller learning rates eventually reach a similar level but take much longer.

enough that the exact accuracy fluctuates by a percent or two. We did not investigate further, but it is worth flagging because it makes the margin over KNN look slightly more favourable than it probably is. KNN's training accuracy (81.1%) sits above its test accuracy (79.8%) in the expected direction, and the gap is small, which is consistent with $K = 10$ being a reasonable bias-variance trade-off.

Cost. The three methods spend compute in different places. Linear regression is basically free: one pseudoinverse during fit, a matrix-vector product at prediction. Logistic regression pays upfront during training (gradient descent over the full set), but prediction afterwards is just as cheap. KNN is the opposite. Fit time is trivial because it only stores the data, but every prediction rebuilds a full $N_{\text{train}} \times N_{\text{test}}$ distance matrix. On a bigger test set this would hurt.

Limitations and future work. Using a single validation split is a fine first pass, but k-fold cross-validation would give hyperparameter estimates with less variance. We also did not expand the features (polynomial or interaction terms), which is the obvious next step if we want the linear models to catch non-linear structure. For KNN, learned distance metrics or per-feature weighting would be the analogue. Both fit into Milestone 2.